

COMS W4156 Advanced Software Engineering (ASE)

September 21, 2023



Please Update Your Team Formation Submissions!

If your team changed in any way, please update your submission for the [team formation](#) assignment. If you did not submit at all, please submit. We need to finalize 'project groups' in courseworks.



Agenda

1. Client-Service Communication Patterns
2. gRPC Overview
3. demo: Yuanhan Tian

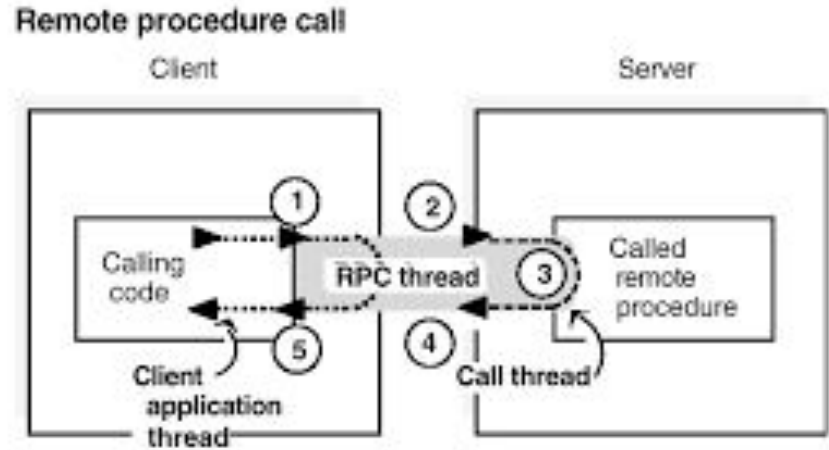


Figure 6-1 Execution Phases of an RPC Thread

Computing

One computer



Caller makes a regular function call to the callee, providing parameters if required by the function signature, e.g., `int foo (int x, int y)`; requires passing two integers

The callee (foo) receives the parameters, performs some computation, and returns result if required by the function signature, e.g., `int foo (int x, int y)`; requires returning an `int`



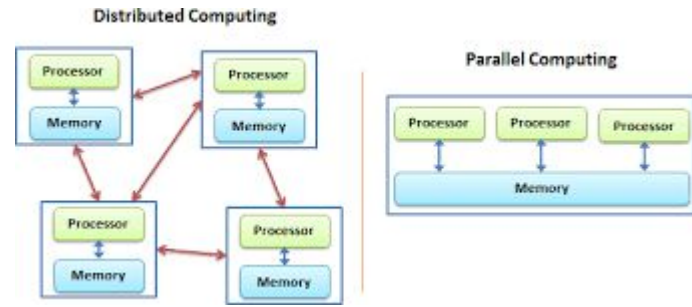
Distributed Computing

Two or more computers

Caller makes what looks like a regular function call to the callee, providing parameters if required by the function signature, e.g., `int foo (int x, int y)`; requires passing two integers

The callee (foo) receives the parameters, performs some computation, and returns what looks like a regular result if required by the function signature, e.g., `int foo (int x, int y)`; requires returning an int

Magic happens under the covers



What Magic?



Client has to find *binding* = naming+locating server

Client parameters need to be *serialized* to send between machines - parameter types restricted to those supported by serialization model (e.g., no pointers)

And then indeed sent via *middleware* 'plumbing' (***Outside scope of this course, if interested look at [lectures from Prof. Geambasu's Distributed Systems course](#)***)

Connection has to be made at server and specific function at server determined

Parameters need to be *deserialized* and that function called with the parameters

Server result has to be serialized and sent back to client via middleware, to be deserialized as return value - return types restricted by serialization model

What Could Possibly Go Wrong?

Server is down or unreachable

- Hang forever waiting for reply
- Time out and raise exception to caller
- Time out and retry

Possible failures worse if request and/or response require multiple network packets not just one

WARNING

The following article contains
instructions for doing something stupid.
Don't try this at home.

Time out and retry sounds best, but lots more things could go wrong:

- Server receives request, performs computation with side-effects, crashes before sending response
- Server restarts and gets request again, but the computation is not idempotent (exactly once vs. at most once vs. at least once)
- Server restarts and gets request again, but it's rejected because of anti-replay semantics

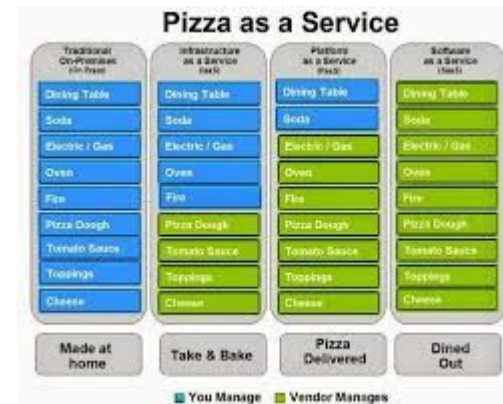
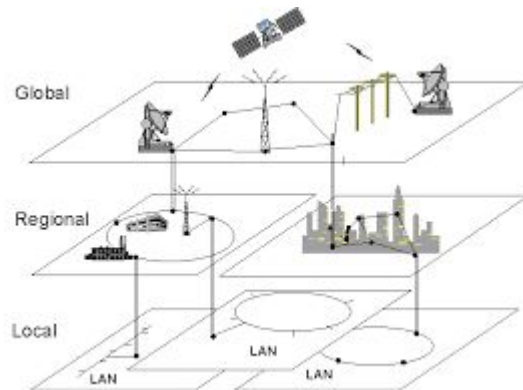
Client-Service Communication Architectures

Client-server or N-tier request-response

Broker or gateway mediated request-response

Service-oriented architecture (SOA) - most clients are themselves services

Publish-subscribe or event-driven push or pull

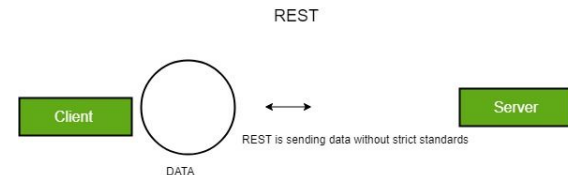
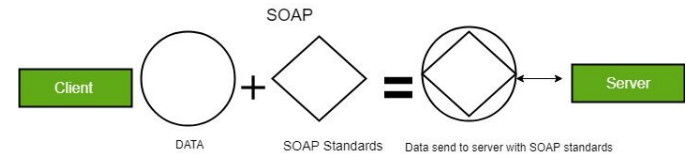
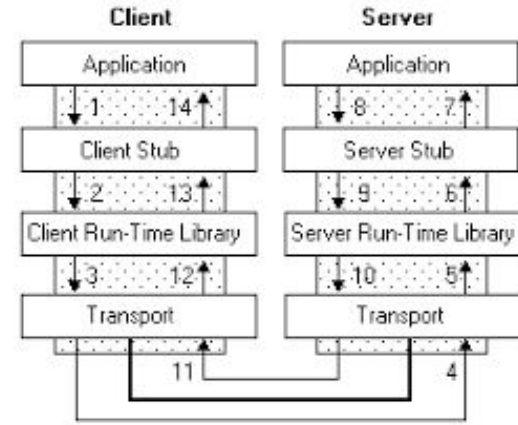


Client-Server Request-Response

Client-server (or N-Tier) messages: send a request, wait for response message

Remote procedure call (RPC):

sending/receiving the message looks like a normal function call, except need to wait while runtime uses some network transport protocol and handle exceptions



What is SOAP?

[Simple Object Access Protocol](#) is a 'standard' for XML-format RPC, alternatives include [Thrift](#), [XML-RPC](#), [JSON-RPC](#)

Data sent back and forth serialized into [json](#), [xml](#), or another predefined language-agnostic format

Pre-XML predecessors [CORBA](#), [COM](#), [RMI](#)

Does anyone still use any of these instead of REST? Yes, [RPC is not dead](#)

2020

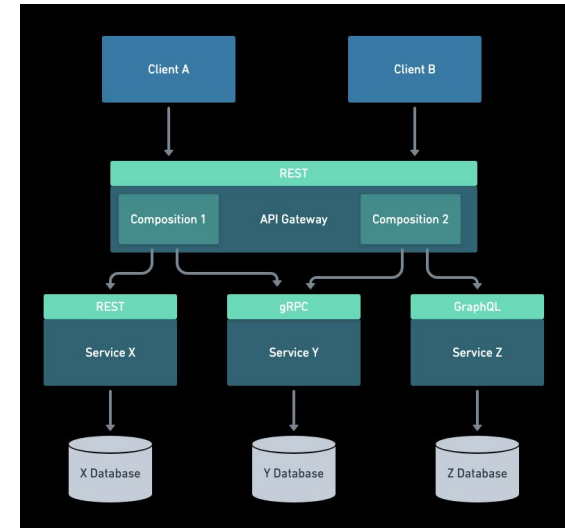
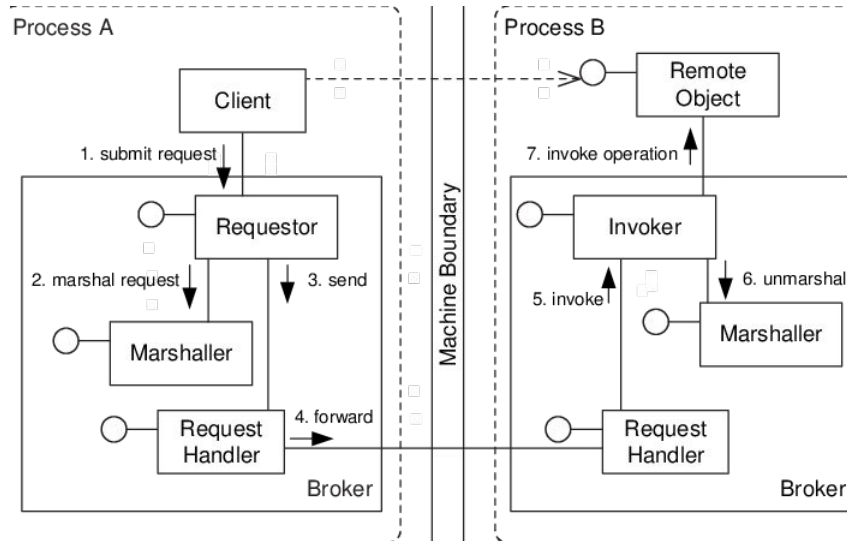
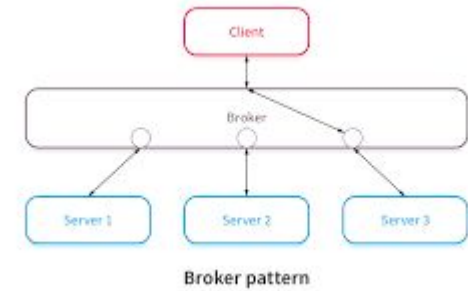


2021



Broker Mediated Request-Response

From client perspective, essentially same as client-server except the client does not know where the server is and binds instead to a [broker](#) or [gateway](#)



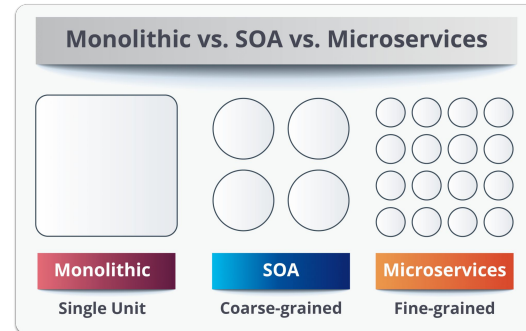
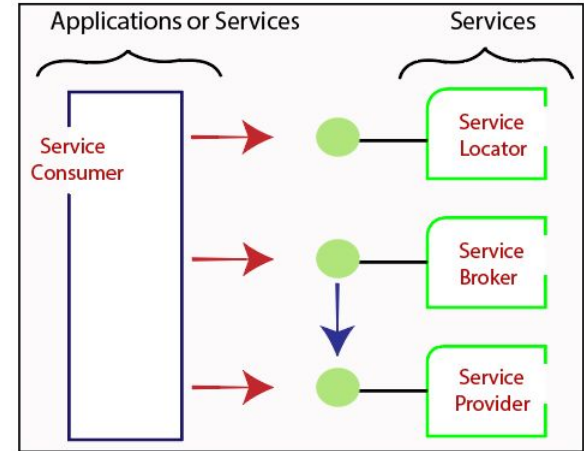
Service-Oriented Architecture

Services interact with other services and with applications over the network

May use both REST and RPC, may use brokers and gateways

Somewhere there's one or more clients (that are not themselves services) for this collection of services, which to the clients look like N-tier

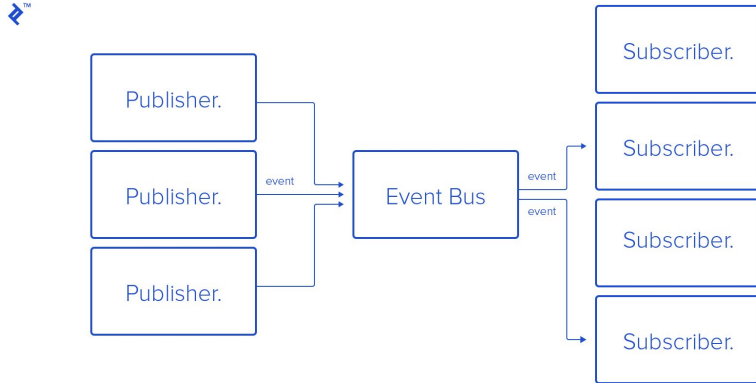
Not much difference between SOA and microservices, other than “SOA” sounds stodgy and “microservices” sounds cool (and SOA generally 'bigger')



Publish-Subscribe Is Different

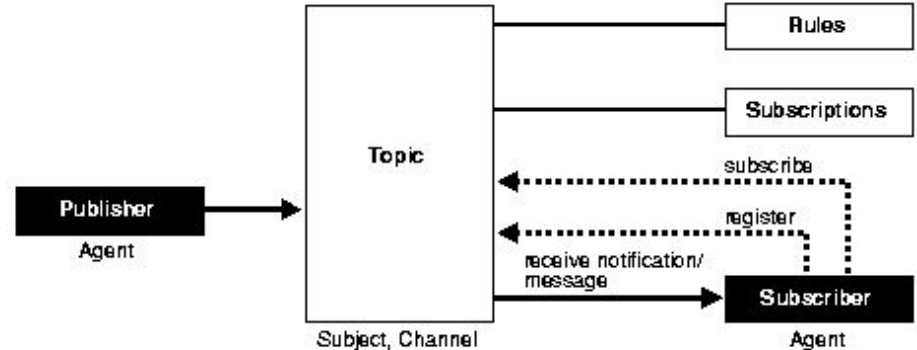
Pub/Sub ~= Twitter for software, not humans

aka Event-Driven Architecture



Push - publisher (or intermediary) sends notification message or invokes callback trigger for “interested” subscribers

Pull (or polling) - Subscribers keep checking

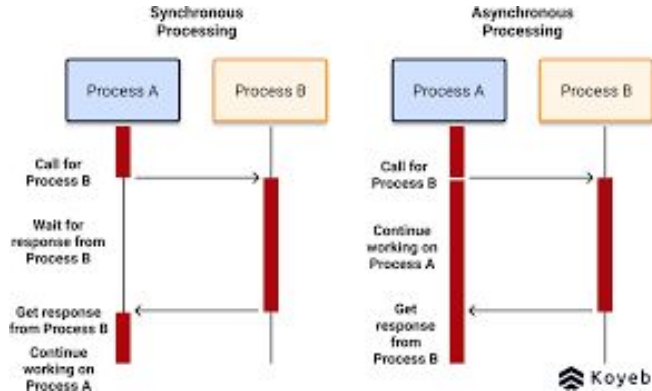


RPC vs. Publish-Subscribe

RPC is (usually) synchronous like a telephone call

Client waits for response from server just like a regular in-process function call

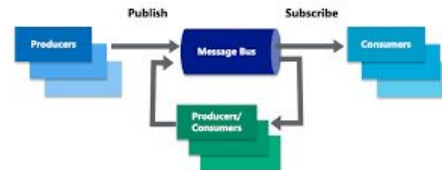
But there needs to be code somewhere to handle networking issues



Publish-subscribe is (usually) asynchronous like a text message

Client subscribes to everything from a specific server or everything that matches a specific pattern from a collection of servers

Client may or may not eventually get message(s) from server(s) and may be deluged with many messages

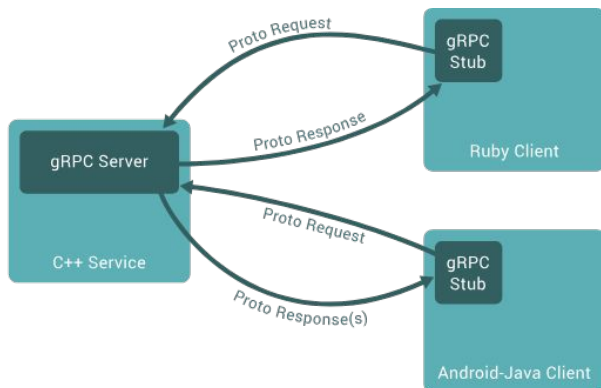


Agenda

1. Client-Service Communication Patterns
2. gRPC Overview
3. demo: Yuanhan Tian

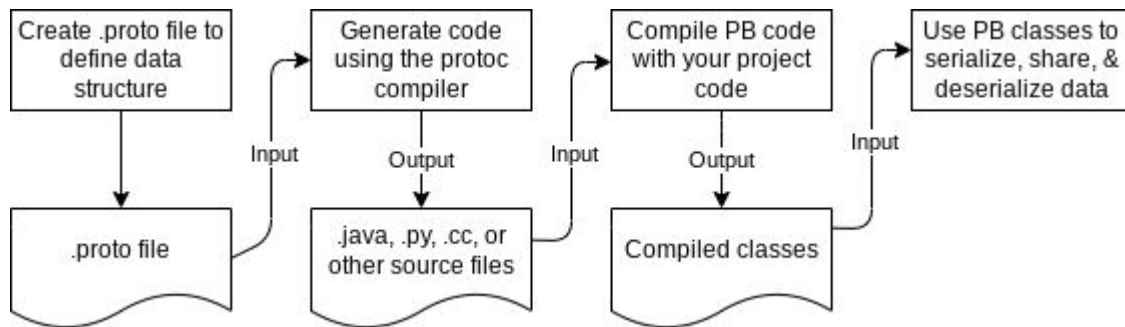


gRPC



Like many other RPC (and REST and Pub/Sub) frameworks, clients and services can be [written in different programming languages and run on different platforms](#)

Clients and services communicate using '[protocol buffers](#)' for serializing structured data, where the proto compiler generates client and service code



gRPC Service Definition Models

1. Unary - single request and single response

```
rpc SayHello (HelloRequest) returns (HelloResponse);
```

2. Server streaming - single request, stream of responses

```
rpc LotsOfReplies (HelloRequest) returns (stream HelloResponse);
```

3. Client streaming - stream of requests, single response

```
rpc LotsOfGreetings (stream HelloRequest) returns (HelloResponse);
```

4. Bidirectional streaming - streams operate independently and may interleave, with order of messages preserved in each stream

```
rpc BidiHello (stream HelloRequest) returns (stream HelloResponse);
```

gRPC Networking Issues

Deadlines (a fixed point in time) and timeouts (durations of time) may succeed on the server side (“I have sent all my responses!”) but fail on the client side (“The responses arrived after my deadline!”)

Services should specify the longest default deadline they technically support, and clients should wait until the response is no longer useful to them

Deadlines can be [adjusted](#) and client requests can be [cancelled](#)

gRPC is [open-source](#) and [well-documented](#)

[What does the 'g' stand for? It's not google](#)

Agenda

1. Client-Service Communication Patterns
2. gRPC Overview
3. gRPC demo: Yuanhan Tian



gRPC demo: Yuanhan Tian

<https://github.com/tiany7/grpc-demo/tree/master>

Upcoming Assignments

[Team Preliminary Project Proposal](#) due next Monday, September 25, by 11:59pm

[Team Revised Project Proposal](#) due a week later Monday, October 2, by 11:59pm

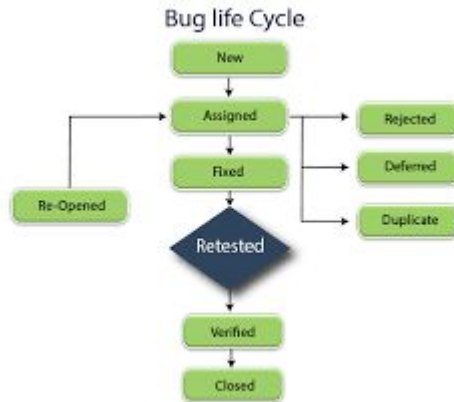
[First Iteration](#) due Monday, October 16, by 11:59pm



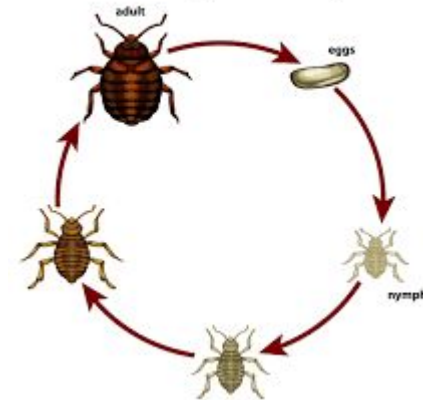
Next Week

Return to (execution-based) bug finding

More on APIs, documenting APIs, and testing APIs



Bed Bug Life Cycle



Please Update Your Team Formation Submissions!

If your team changed in any way, please update your submission for the [team formation](#) assignment. If you did not submit at all, please submit. We need to finalize 'project groups' in courseworks.



Ask Me Anything